



Arun Bhat <meetarunkumar@gmail.com>

Seva mitra developmet plan

1 message

Prasanna kumar <bv.prasannakumar@gmail.com>
To: Arun Samartha <meetarunkumar@gmail.com>

17 September 2026 at 04:49

Sir Namaste,
Chat GPT has given this plan

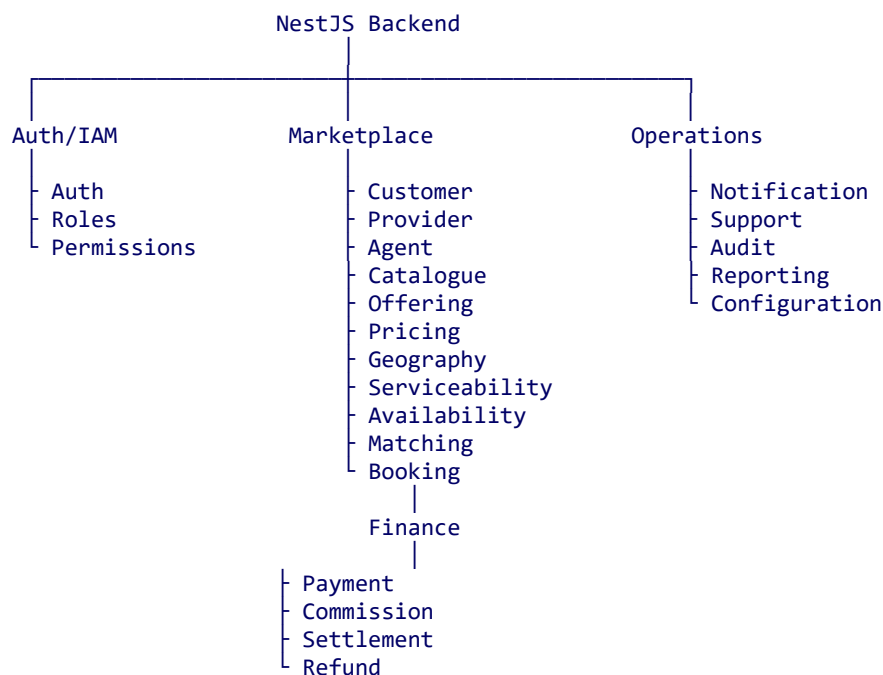
For SevaMitra, I'm planning the backend as a **NestJS modular monolith initially**. These are real domain modules, not microservices yet. Later, high-load modules can be extracted without redesigning the whole platform.

Proposed SevaMitra backend modules

#	Backend Module	Main Responsibility	Pilot
1	Auth	Login, OTP, password, social login, tokens, logout, sessions	✓
2	IAM / Role Management	Users, dynamic roles, permissions, scopes, role assignments	✓
3	Customer	Customer profile, addresses, preferences, account lifecycle	✓
4	Provider	Individual providers, profiles, verification, status	✓
5	Provider Organization	Provider companies, groups, staff/memberships	✓ Basic
6	Agent	Individual agents, agent companies, attribution, incentives	✓
7	Provider Onboarding	Agent/self onboarding, documents, approval/rejection, bulk onboarding	✓
8	Service Catalogue	Categories, services, variants, configurable attributes	✓
9	Provider Offering	Which provider offers which services	✓
10	Pricing & Quote	Provider prices, price rules, estimates/quotes	✓
11	Geography	Country/state/district/taluk/town/village/PIN masters	✓
12	Serviceability	PIN/radius/polygon coverage, provider coverage matching	✓
13	Availability	Provider working hours, leave, exceptions, bookable availability	✓
14	Discovery / Matching	Find providers by service, location, availability and other criteria	✓
15	Booking	Service request, booking, assignment, lifecycle, cancellation	✓
16	Payment	Payment intent, transaction, gateway integration, payment status	✓
17	Commission	Platform/provider/group/category commission rules and calculations	✓
18	Settlement	Provider earnings, payout calculations, settlement tracking	✓
19	Refund	Refund requests, approval, processing, reversal	✓ Basic
20	Promotion / Coupon	Offers, campaigns, coupons, eligibility	Phase 2
21	Wallet / Cashback	Cashback credits/debits and wallet ledger	Phase 2
22	Loyalty / Referral	Points, referrals and rewards	Phase 2
23	Review & Rating	Customer reviews, ratings, provider rating summary	✓
24	Notification	Push, SMS, email, WhatsApp orchestration/templates	✓
25	Support	Customer/provider support tickets and case management	✓ Basic
26	Dispute	Booking/payment/provider disputes and evidence	Phase 2
27	Safety / Incident	Safety/emergency/property-damage cases	Phase 2
28	Fraud / Risk	Abuse detection, suspicious accounts/bookings/referrals	Later
29	Configuration	Dynamic business/platform configuration	✓
30	Theme / Localization	Dynamic themes, language resources, regional settings	✓ Basic
31	Media / Document	Images, provider documents, verification evidence, uploads	✓
32	Audit	Who changed what, when, where and why	✓
33	Reporting / Dashboard	Customer/provider/agent/admin operational reports	✓ Basic
34	Event / Outbox	Reliable domain events and future Kafka integration	✓
35	Scheduler / Jobs	Expiry, reminders, retries, settlement jobs, housekeeping	✓ Basic

There are **35 logical modules**, but this absolutely does **not** mean 35 applications or 35 microservices.

Our pilot backend remains roughly:



A few boundaries are particularly important

Auth and Role Management should be separate. Auth answers *"Who are you?"* IAM answers *"What are you allowed to do, and where?"* This matters because our admins/operators aren't based on a few hard-coded roles.

For example:

User: Prasanna

Role:
Regional Operations Manager

Permissions:
provider.view
provider.approve
booking.view
booking.reassign
refund.view

Scope:
Assigned Geography

Tomorrow we could create Verification Manager without changing JavaScript/TypeScript code simply by combining different permissions.

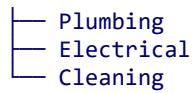
Service Catalogue and Provider Offering should also be separate.

This is critical for our open-service model:

SERVICE CATALOGUE

Beauty & Wellness
 └─ Makeup
 └─ Mehendi
 └─ Hair Styling

Home Services
 |

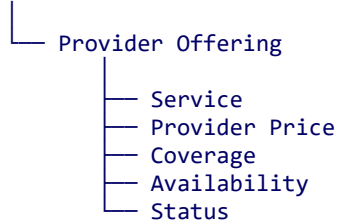


Spiritual Services



Then:

Provider



Therefore adding a completely new legitimate service later doesn't require us to create a new backend module.

Pricing, Commission and Payment are three different domains.

They shouldn't be mixed.

Provider Pricing

"What does provider charge?"



Booking Price

"What did customer agree to?"



Payment

"What did customer actually pay?"



Commission

"What does SevaMitra earn?"



Provider Earning

"What does provider receive?"



Settlement

"Has that money actually been paid out?"

This separation will save us a lot of trouble later.

Geography and Serviceability are also different.

Geography stores:

State
District
Taluk
Town
Village
PIN
Coordinates

Serviceability determines:

Can electrician ABC
provide Service XYZ
at customer location X?

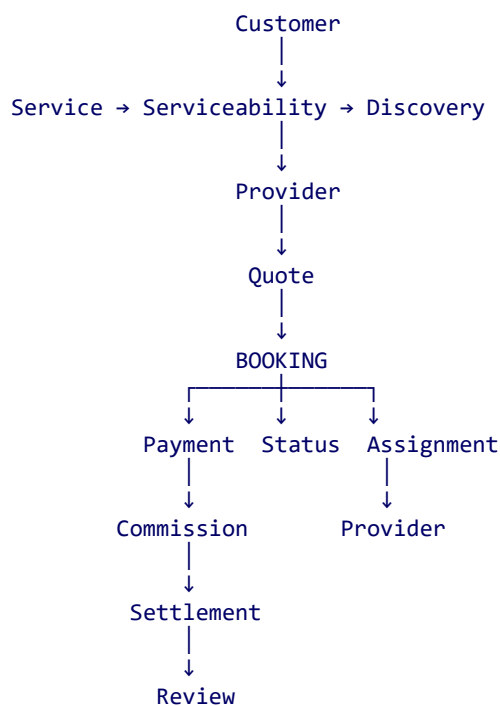
It may evaluate:

PIN
+
radius
+
polygon
+
service
+
provider coverage
+
provider status
+
availability

That is why I don't want serviceability logic scattered throughout Provider and Booking.

Booking will be the heart of SevaMitra

A lot of modules eventually converge here:

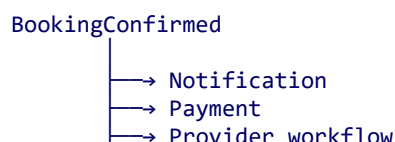


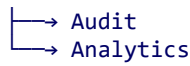
Because of that, I want **Booking to remain its own strong domain module**, rather than putting booking methods inside Customer or Provider.

And one more important module: Event/Outbox

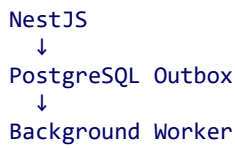
Even though we are **not using Kafka in the pilot**, we should design business events from day one.

For example:

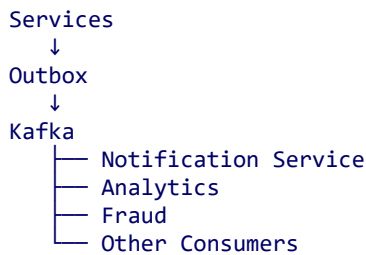




Initially:



Later, when scale actually requires it:



So we get future scalability **without paying for Kafka/Kubernetes today**.

What I would actually code first

Although the architecture recognizes ~35 domains, our **first backend milestone should contain only about 14 core modules**:

Auth → IAM → Customer → Provider → Agent → Provider Onboarding → Catalogue → Geography → Serviceability → Offering/Pricing → Availability → Booking → Payment → Audit/Notification.

Once one complete journey works:

Customer registers → finds a service → finds a provider → books → provider accepts → completes → customer pays → gets notification → reviews

then we add **commission/settlement, provider companies, promotions, support, reports, advanced agent functionality, loyalty, fraud, etc.**

That gives us a much safer development path than creating 35 empty NestJS modules on day one.